CCCM Disaster Coordination: Project Plan

MERN web app (MongoDB, Express, Node, React) for a 2-person semester project. Covers scope, application flow, database structure, site assessment APIs, LLM use, and the build schedule.

1. What we are building

A coordination platform for disaster relief camps in Pakistan.

- NGOs register, get admin approval, and choose for themselves which disaster to help with.
- Camp managers assess a site using NDMA standards, create camps, and post shortages.
- Affected people submit requests and complaints. The AI tags urgency and NGOs are emailed.
- Admin oversees accounts and disasters. Admin does **not** assign NGOs.

Teacher's required features

#	Feature	Where it lives
1	Email verification (6-digit code) + JWT login	Login and register
2	Four roles: Admin, NGO, Camp Manager, Affected Person	Role-based middleware
3	Complaints and needs from affected people	Complaints page
4	Email to NGOs when a need is raised	Complaints page

AI integration is optional, so our LLM features are the extra credit.

Scope

- **Required:** the four features above.
- **Core:** site assessment + NDMA calculator, camp and shortage management, NGO self-commitment.
- **AI extras:** LLM site report, complaint urgency tagging.

- **Dropped:** RAG assistant, dashboards, report-generation page.

Who owns what

Area	Owner
Login, email verification, roles, admin approvals	Partner
Disasters, NGO commitments	Partner
Camp creation, management, closure, resource levels	Partner
Site assessment, calculator, map APIs, LLM report	You
Complaints, urgency tagging, NGO emails	You

2. Application flow

1. **NGO registers** with email code verification. Account status is `pending`.
2. **Admin approves** the NGO. Status becomes `active`.
3. **Admin creates a disaster.** Approved NGOs see it and click "I will respond", entering work area, activities and resources.
4. **Camp manager assesses a site:**
   - Picks a location on the map (pin or coordinates).
   - Server fetches indicative map data (river distance, elevation, hospitals, roads).
   - Manager fills the form facts (area, population, slope, road, water, flood history).
   - Rules engine + calculator return `suitable`, `needs_review` or `not_suitable` with NDMA page references.
   - LLM writes a short readable report from that data.
5. **Camp is created** with capacity, population, facilities and a planned closure date.
6. **Manager updates shortages** (water, food, medicine: sufficient / low / critically low). All approved NGOs see them and can offer help.
7. **Affected person submits a request.** AI tags category and urgency. Relevant NGOs are emailed. Protection concerns go to the manager only.
8. **At the closure date**, the manager confirms or extends. Camps never close automatically.

3. Database structure (MongoDB collections)

Use Mongoose models and ObjectId references. Build the first six early, the rest as each feature starts.

users

One collection for all four roles.

Field	Type	Notes
_id	ObjectId	
fullName	String	
email	String	unique
passwordHash	String	bcrypt
role	String	admin, ngo, campManager, affectedPerson
isEmailVerified	Boolean	
accountStatus	String	pending, active, suspended, rejected
createdAt, updatedAt	Date	

emailVerificationCodes

Field	Type	Notes
userId	ObjectId	ref users
codeHash	String	hashed 6-digit code
expiresAt	Date	TTL index
attempts	Number	limit failed tries

ngos

Field	Type	Notes
userId	ObjectId	ref users
organizationName	String	
description, contactNumber	String	
operatingAreas	[String]	used for email matching
services	[String]	food, water, medicine, shelter
approvalStatus	String	pending, approved, rejected
approvedBy	ObjectId	admin

disasters

Field	Type	Notes
title, type, description	String	
location	Object	district, province, optional lat/lng
severity	String	low, medium, high, critical
affectedPopulation	Number	
status	String	active, underControl, closed
createdBy	ObjectId	admin

disasterCommitments

Unique compound index on `disasterId + ngoId`.

Field	Type	Notes
disasterId, ngoId	ObjectId	
workAreas, plannedActivities	[String]	

Field	Type	Notes
resourcesOffered	[{type, quantity}]	
status	String	committed, active, withdrawn, completed

siteAssessments (yours)

Field	Type	Notes
managerId	ObjectId	ref users
disasterId	ObjectId	optional
location	Object	name, lat, lng
input	Object	area, population, slope, allSeasonRoad, waterSourceNearby, floodRiskManager, landslideRisk, malariaProne, titleClear
apiData	Object	riverDistanceM, elevationM, nearestHospitalKm, floodDischarge (all marked indicative)
result	String	suitable, needs_review, not_suitable
fails, warns	[String]	with NDMA page refs
requirements	Object	output of calculateRequirements
llmReport	String	
createdAt	Date	

camps

Field	Type	Notes
name	String	
managerId	ObjectId	ref users
disasterId	ObjectId	

Field	Type	Notes
siteAssessmentId	ObjectId	
location	Object	
capacity, population	Number	
facilities	[String]	
resources	[{name, level}]	level: sufficient, low, critical
plannedClosureDate	Date	
status	String	active, closureDue, extended, closed

campNgos

Which NGOs work at which camp.

Field	Type	Notes
campId, ngoId	ObjectId	
focus	String	e.g. Water
joinedAt	Date	

shortageOffers

Field	Type	Notes
campId, ngoId	ObjectId	
resource	String	
message	String	
status	String	offered, accepted, declined

requests (complaints, yours)

Field	Type	Notes
userId	ObjectId	affected person
campId	ObjectId	
category	String	food, water, medicine, shelter, problem, protection
details	String	
isRestricted	Boolean	true for protection; server hides from NGOs
aiUrgency	String	low, medium, high, critical
aiCategory	String	
status	String	pending, in_progress, resolved
createdAt	Date	

notifications

Log of emails sent (for debugging and the demo).

Field	Type	Notes
requestId	ObjectId	
ngoId	ObjectId	
sentAt	Date	
status	String	sent, failed

4. Site assessment: how it works

Three layers. **The LLM never decides suitability.**

Layer	Source	Purpose
Location	Leaflet + OpenStreetMap map, pin drop	Gives lat/lng

Layer	Source	Purpose
Site facts	Manager form + free map APIs	Inputs to rules
Decision	`guidelines.js` rules engine + calculator	suitable / needs_review / not_suitable
Write-up	LLM API	Readable report from the JSON

APIs used (all free, check availability before committing)

API	Gives us	Used for
Leaflet + OpenStreetMap tiles	Map and pin	Location input
Nominatim (OSM geocoding)	Coordinates from a place name	Typing "Peshawar"
Overpass API (OSM data)	Nearest river/canal, hospitals, roads	River distance, hospital km, road check
Open-Meteo Flood API	River discharge forecast	Indicative flood signal
Open-Meteo Elevation API	Elevation at a point	Low-lying check

All values from these APIs are labelled **indicative** and the manager confirms them. They are rough signals, not a flood-risk map.

Rules (from NDMA 2017, in guidelines.js)

Check	Rule	Result if broken
Area	30 m² per person minimum (45 comfortable), p.6	fail below 30, warn below 45
Slope	1-5%, p.4	warn
Road	all-season access, p.4	fail
Water source nearby	p.4	warn

Check	Rule	Result if broken
Landslide risk	p.4	fail
Malaria / water-borne disease	p.4	warn
Land title	clear, p.4	warn
Flood risk (manager)	high = fail, medium = warn	
River distance	indicative only, not NDMA	warn

Calculator outputs for N people

Land, covered shelter (3.5 m² each), tents, water (15-20 L/person/day), taps, hand pumps, hand basins, toilets (1 per 20), waste pits, soap, ambulance (over 1,000 people), watch towers.

Items to verify before the demo

- **River distance thresholds** (500 m / 1,000 m) are our placeholders, not NDMA. Label them "project threshold".
- **Family size of 6** is a placeholder.
- **Latrine distance from water**: p.10 says 30 m, p.9 says 20 ft. Check the original.
- **Pit above water table**: PDF reads "1-5 meters", likely 1.5. Check the original.

5. LLM use (kept simple)

Feature	Input	Output
Site report	Assessment JSON + apiData	Under 250 words: Summary, Risks, Resource needs, Recommended actions. Uses only provided data, reports `result` exactly as given
Complaint urgency	Complaint text	category + urgency (low/medium/high/critical) as JSON

Guardrails:

- Validate the JSON response. If the LLM fails, store the complaint untagged and carry on.
- Never block a complaint on an LLM error.
- Keep the API key in `.env` on the server only.

6. Email notifications

- Library: Nodemailer (Gmail app password or a free SMTP such as Brevo).
- **Verification:** 6-digit code, hashed, expires in 10 minutes, 5 attempts max.
- **New request:** email approved NGOs whose `operatingAreas` or camp links match the request's camp. Skip restricted protection complaints.
- Log each email in `notifications`.

7. Access control (enforced on the server)

Role	Can do
Admin	Approve NGOs, manage accounts, create disasters
NGO	View disasters, commit, view shortages, offer help, view non-restricted requests for their camps
Camp Manager	Assess sites, create and manage camps, update shortages, view all requests including protection
Affected Person	Submit and track own requests, view camp info

Protection complaints are filtered in the database query, not only hidden in the UI.

8. Folder structure

```
cccm-project/
├── client/src/ (components, pages, services, context, App.jsx)
├── server/src/
|   ├── config/ (db.js, guidelines.js)
|   ├── models/ routes/ controllers/ middleware/
|   ├── services/ (mapData, llm, email)
|   └── app.js
├── .gitignore
└── README.md
```

9. Main API endpoints

Area	Endpoint
Auth	POST /auth/register, /auth/verify, /auth/login
Admin	GET /admin/ngos, PATCH /admin/ngos/:id, POST /disasters
Disasters	GET /disasters, POST /disasters/:id/commit
Site assessment	POST /sites/map-data (fetch APIs), POST /sites/assess (rules + calculator), POST /sites/:id/report (LLM)
Camps	POST /camps, PATCH /camps/:id, PATCH /camps/:id/resources, POST /camps/:id/closure (confirm or extend)
Shortages	POST /camps/:id/offers
Requests	POST /requests, GET /requests, PATCH /requests/:id/status

10. Project plan

Week	You	Partner
1	Repo, setup, MongoDB, siteAssessment model, agree API formats	Repo, setup, User model, auth, email verification
2	Port `guidelines.js`, `/sites/assess` endpoint, site form UI	Role middleware, NGO profiles, admin approvals
3	Leaflet map, Overpass + Open-Meteo calls, `/sites/map-data`	Disasters list, NGO commitments
4	LLM site report, assessment results page	Camp creation, edit, closure, NGO links
5	Requests model, submit and track UI, restricted access	Resource levels, shortage offers
6	LLM urgency tagging, NGO email notifications	Integrate camp flow with site assessment ("Create camp")
7	Integration, Postman and UI testing, bug fixes	Integration, testing, bug fixes
8	Documentation, demo data, final demo rehearsal	Documentation, deployment, final demo rehearsal

Milestones

- **End week 2:** login works, roles enforced, rules engine returns a result.
- **End week 4:** full flow from NGO approval to camp creation.
- **End week 6:** all four required features and both AI extras working.
- **End week 8:** tested, documented, demo-ready.

Team rules

- Same field names in forms and schemas.
- ObjectId references everywhere.
- DB logic in models/services, request handling in controllers.
- Separate Git branches, merge through pull requests.

- Test every API in Postman before connecting its React page.
- `.env` never committed.

Risks and fallbacks

Risk	Fallback
Overpass or Open-Meteo is slow or down	Manager enters river distance and elevation manually; rules still run
LLM API fails or is rate limited	Show the rules result without the report; complaints saved untagged
Email provider limits	Log emails to console and `notifications` for the demo
Schedule slips	Cut the LLM report first, keep the four required features

11. Build order if time is short

1. Four required features (auth + email code, roles, complaints, NGO emails)
2. Camps, disasters, NGO commitments, shortages
3. Site assessment with calculator
4. Complaint urgency tagging
5. Map APIs
6. LLM site report